Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Лабораторная работа №5

«Модульное тестирование в Python»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-32Б _____ (Подругин Г.Р.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Цель и задание

Проверить фрагмент первой лабораторной работы с помощью тестового фреймворка, BDD-сценариев и Mock-объектов.

Выбран процедурный решатель биквадратного уравнения. Требуется не менее трёх тестов TDD-фреймворка и не менее трёх тестов BDD-фреймворка. Дополнительно реализована проверка повторного ввода с Mock.

Описание решения

Функция solve принимает коэффициенты и возвращает объект Solution без чтения ввода. Функция read_coefficients принимает зависимости input_fn и output_fn, поэтому взаимодействие с консолью заменяется Mock. pytest проверяет четыре корня, отсутствие корней, тождество и повтор ошибочного ввода.

Три сценария Gherkin используют шаги Given, When и Then. pytest-bdd связывает шаги с функциями и общими фикстурами коэффициентов и результата. Mock задаёт последовательность ответов и позволяет проверить количество вызовов. Сеть и реальная клавиатура в тестах не используются. Общие модули подключаются из lab_01 через pytest.ini.

Порядок запуска

Python 3.10 или новее. Из корня репозитория создать виртуальное окружение и установить зависимости командой python -m pip install -r requirements.txt.

```
python -m pytest lab_05 -v
```

Результаты выполнения

```
$ python -m pytest lab_05 -v --tb=short
============================= test session starts ==============================
configfile: pytest.ini
plugins: bdd-8.1.0
collecting ... collected 7 items

lab_05/test_bdd.py::test_four_distinct_real_roots PASSED                 [ 14%]
lab_05/test_bdd.py::test_no_real_roots PASSED                            [ 28%]
lab_05/test_bdd.py::test_every_real_number_is_a_solution PASSED          [ 42%]
lab_05/test_tdd.py::test_four_roots PASSED                               [ 57%]
lab_05/test_tdd.py::test_no_real_roots PASSED                            [ 71%]
lab_05/test_tdd.py::test_degenerate_identity PASSED                      [ 85%]
lab_05/test_tdd.py::test_bad_input_with_mock PASSED                      [100%]

============================== 7 passed in 0.02s ===============================
```

Рисунок 1 — Вывод фактического запуска программы

Экранная форма сформирована из сохранённого stdout. Полные журналы: results/demo.txt и results/tests.txt.

Проверка и выводы

Для этой работы успешно выполнено проверок: 7. Общий набор всех шести работ содержит 61 тест; полный прогон завершился без ошибок. Версии среды и журналы сохранены в validation/ и results/tests.txt.

Выполнены четыре теста pytest и три BDD-сценария. Все семь проверок завершились успешно. Разделение вычислений и ввода позволило тестировать математическую логику и восстановление после ошибочного ввода независимо.

Источник задания

https://github.com/ugapanyuk/BKIT_2022/wiki/lab_python_test

Текст программы

```
# Файл test_tdd.py

"""Тесты функций лабораторной 1 с использованием pytest."""
from unittest.mock import Mock
import pytest
from procedural import solve
from common import read_coefficients


def test_four_roots():
    assert solve(1, -5, 4).roots == pytest.approx((-2, -1, 1, 2))


def test_no_real_roots():
    assert solve(1, 0, 1).roots == ()
    assert not solve(1, 0, 1).infinite


def test_degenerate_identity():
    assert solve(0, 0, 0).infinite


def test_bad_input_with_mock():
    reader = Mock(side_effect=["wrong", "1"])
    writer = Mock()
    assert read_coefficients(["bad", "-5", "4"], reader, writer) == (1, -5, 4)
    assert reader.call_count == 2
    assert writer.call_count == 2

# Файл features/equation.feature

Feature: Real roots of a biquadratic equation
  Scenario: Four distinct real roots
    Given coefficients 1, -5, 4
    When I solve the equation
    Then the roots are -2, -1, 1, 2

  Scenario: No real roots
    Given coefficients 1, 0, 1
    When I solve the equation
    Then there are no real roots

  Scenario: Every real number is a solution
    Given coefficients 0, 0, 0
    When I solve the equation
    Then every real number is a solution

# Файл test_bdd.py

import pytest
from pytest_bdd import given, when, then, scenarios, parsers
from procedural import solve

scenarios("features/equation.feature")
```

Текст программы Продолжение

```
@given(parsers.parse("coefficients {a:g}, {b:g}, {c:g}"),
    target_fixture="coefficients")
def coefficients(a, b, c):
    return a, b, c


@when("I solve the equation", target_fixture="solution")
def solve_equation(coefficients):
    return solve(*coefficients)


@then("the roots are -2, -1, 1, 2")
def four_roots(solution):
    assert solution.roots == pytest.approx((-2, -1, 1, 2))


@then("there are no real roots")
def no_roots(solution):
    assert solution.roots == () and not solution.infinite


@then("every real number is a solution")
def infinite_roots(solution):
    assert solution.infinite
```

```
# Файл ../lab_01/common.py

"""Общий ввод и представление результата трёх реализаций."""
import math
import sys
from dataclasses import dataclass


@dataclass(frozen=True)
class Solution:
    roots: tuple[float, ...] = ()
    infinite: bool = False


def read_coefficients(args, input_fn=input, output_fn=print):
    values = []
    for index, name in enumerate("ABC"):
        raw = args[index] if index < len(args) else None
        while True:
            if raw is None:
                raw = input_fn(f"{name} = ")
            try:
                value = float(raw)
                if not math.isfinite(value):
                    raise ValueError
                values.append(value)
                break
            except (ValueError, TypeError):
                output_fn(f"Некорректный коэффициент {name}. Введите конечное
    число.")
                raw = None
    return tuple(values)
```

Текст программы Продолжение

```
def run(solver):
    try:
        a, b, c = read_coefficients(sys.argv[1:])
        result = solver(a, b, c)
        print(f"A = {a:g}; B = {b:g}; C = {c:g}")
        print(f"Дискриминант = {b * b - 4 * a * c:g}")
        if result.infinite:
            print("Решение: любое действительное число")
        elif result.roots:
            print("Действительные корни:", ", ".join(f"{x:.12g}" for x in
    result.roots))
        else:
            print("Действительных корней нет")
    except (EOFError, KeyboardInterrupt):
        print("\nВвод прерван", file=sys.stderr)
        return 1
    except (ValueError, OverflowError) as error:
        print(f"Ошибка вычисления: {error}", file=sys.stderr)
        return 1
    return 0

# Файл ../lab_01/procedural.py

"""Процедурное решение A*x**4 + B*x**2 + C = 0."""
from math import copysign, sqrt
from common import Solution, run


def solve(a, b, c):
    if a == 0:
        if b == 0:
            return Solution(infinite=c == 0)
        squares = (-c / b,)
    else:
        # Масштабирование снижает вероятность переполнения дискриминанта.
        scale = max(abs(a), abs(b), abs(c))
        a, b, c = a / scale, b / scale, c / scale
        d = b * b - 4 * a * c
        if d < 0:
            return Solution()
        if d == 0:
            squares = (-b / (2 * a),)
        else:
            q = -0.5 * (b + copysign(sqrt(d), b))
            squares = (q / a, c / q)
    roots = set()
    for y in squares:
        if y >= 0:
            roots.update((-sqrt(y), sqrt(y)))
    return Solution(tuple(sorted(roots)))


if __name__ == "__main__":
    raise SystemExit(run(solve))
```